

1. Introduction

The Smart Fun Run Tech Event represents an integrated engineering challenge requiring seamless cooperation between physical Internet of Things (IoT) edge nodes and robust Operating System (OS) backend architectures. In a large-scale public event like a fun run, tracking participants, ensuring safety, managing data traffic, and monitoring system statuses are vital for operational success.

To bridge the gap between hardware data acquisition and backend multi-threading servers, a centralized dashboard is crucial. This report outlines the design, implementation, and deployment of a localized Node-RED dashboard that aggregates operational data, visualizes real-time metrics, and provides system administrators with an intuitive interface to monitor the overall health and performance of the Smart Fun Run microservices.

2. Proposal Idea

The proposed solution utilizes Node-RED as the central orchestrator and data visualization hub. The dashboard serves as a unified command center that interfaces with various sub-group microservices via standard data transmission protocols.

System Architecture & Logic

- **Data Ingestion:** The dashboard listens for data payloads transmitted from edge nodes or backend servers, parsing them into standardized JSON formats.
- **Real-Time Visualization:** Received metrics—such as participant checkpoint scan times, network latency, and throughput—are routed to UI nodes for dynamic charting and status indications.
- **Alert Mechanism:** Flow logic is implemented to detect anomalies (e.g., dropped packets, disconnected nodes, or missing updates) and trigger immediate visual notifications for system administrators.

3. Pre-Environment Setup

Setting up Node-RED requires a local environment capable of running JavaScript via runtime engines. Below are the sequential steps to prepare the system environment.

Step 1: Install Node.js

Node-RED is built on top of Node.js, making its runtime environment mandatory.

1. Download the LTS version of **Node.js** from the official website.
2. Run the installer, ensuring that the option to add Node.js to the system environment path variables is checked.
3. Verify the installation by opening a terminal or command prompt and running:

Bash

```
node -v  
npm -v
```

Step 2: Install Node-RED

With Node Package Manager (npm) ready, Node-RED can be installed globally on the local system.

1. Execute the following command in the terminal:

Bash

```
npm install -g --unsafe-perm node-red
```

2. Wait for the compilation and package fetching to finish.

Step 3: Install Dashboard Nodes

By default, Node-RED handles logic flows. Visual elements require the dashboard node package.

1. Launch Node-RED by typing node-red in the terminal.
2. Open a web browser and navigate to <http://127.0.0.1:1880>.
3. Click the menu icon in the top right corner, select **Manage palette**, go to the **Install** tab, search for node-red-dashboard, and click install.

4. Procedure

The construction of the centralized dashboard involves node configuration, message routing, and UI layout design.

Flow Construction Steps

1. **Input Configuration:** Drag and drop communication protocol nodes (such as **MQTT In**, **HTTP In**, or **TCP/UDP In**) into the workspace to act as the entry points for external system payloads.
2. **Data Parsing:** Connect the input node to a **JSON functional node** to automatically convert incoming string payloads into accessible JavaScript objects (`msg.payload`).
3. **Business Logic & Routing:** * Wire a **Switch Node** to check the status or origin of the incoming message.
 - Route specific telemetry data (e.g., network delays or participant counts) to corresponding analytical pathways.
4. **UI Element Mapping:** Wire the parsed data streams directly to **Node-RED Dashboard nodes** (such as *Charts*, *Gauges*, *Text inputs*, and *Audio alerts*).
5. **Deployment:** Click the **Deploy** button in the top right corner of the editor window to compile the flows and make the dashboard live at <http://127.0.0.1:1880/ui>.

5. Result and Analysis

The completed Node-RED dashboard successfully unifies data from independent streams into a single, cohesive graphical user interface.

Performance Analysis

- **Latency:** Data transmission from incoming packets to visual graph updates demonstrates sub-second processing latency, validating Node-RED's event-driven architecture for live tracking.
- **Payload Integrity:** The system successfully parses structured JSON objects without dropping packets under normal simulated load conditions.
- **UI Responsiveness:** The dashboard effectively visualizes metrics using dynamic gauges for system load, real-time scrolling charts for connection health, and status grids for individual sub-group microservices.

6. Troubleshooting

During deployment and system verification, several common integration challenges may occur. The table below highlights the key problems encountered and their resolutions:

Symptom / Error	Root Cause	Corrective Action
Command not found when running node-red	Node.js/npm environment variables are not registered in the system path.	Re-run the Node.js installer and explicitly check the "Add to PATH" option, or manually append the npm directory to system environment variables.
Dashboard UI shows a blank screen or 404 error	The node-red-dashboard package is missing or the flow lacks active UI nodes.	Install the dashboard package via the palette manager and ensure at least one UI component is wired and deployed.
Incoming data fails to display on charts	The incoming payload is formatted as a raw string instead of a valid JSON object.	Insert a JSON node between the input receiver and the dashboard gauge to parse the text into an object structure.
Network port conflicts (Port 1880 already in use)	Another instance of Node-RED or a competing service is holding the default network port.	Kill the existing background process using task manager or modify the listening port in the settings.js configuration file.

7. Conclusion

The implementation of the centralized Node-RED dashboard establishes a robust, highly visible monitoring solution for the Smart Fun Run Tech Event. By successfully parsing network payloads and converting them into instantaneous visual feedback, the platform fulfills the core criteria of system integration between edge networks and backend processing layers. This centralized approach reduces troubleshooting time, provides clear data overview capabilities, and ensures that complex multi-service ecosystems remain stable and verifiable during live operations.